INDEX

FlowZen – AI-First Workflow Automation Platform

Project Title

FlowZen – AI-First Workflow Automation Platform

---

1. Introduction

FlowZen is a large-scale, AI-first workflow automation platform designed and built with a strong focus on correctness, reliability, and real-world production behavior. This project is inspired by popular workflow automation tools such as n8n, Zapier, and Temporal, but it is not a clone or copy. Instead, it is a deeply engineered system that focuses on understanding *how* automation platforms actually work internally, rather than only copying surface-level UI behavior.

The platform allows users to visually design workflows using nodes, connect those nodes to define execution order, and then execute those workflows using a custom backend execution engine. Each workflow can include triggers, logic nodes, AI nodes, and action nodes such as sending emails or replying on Telegram.

Unlike many student or demo projects, this system was developed using a production mindset. The developer intentionally chose to face real-world complexity such as OAuth security, webhook behavior, background workers, Docker failures, database migrations, and execution consistency. This makes the project not only technically rich, but also highly educational and realistic.

This documentation is written in very detailed and simple English. It explains the full journey of the project: why it was started, how it was built, what problems occurred, why those problems happened, and how each problem was solved. The goal is to make the document long, complete, and easy to understand, even for someone who is new to workflow automation systems.

1.1 Why This Project Was Needed

In many automation tools, users trust the UI blindly. If the UI shows that a workflow ran successfully, users assume that emails were sent, messages were delivered, or actions were completed. In reality, many systems fail silently. Emails may not be sent, webhooks may not fire correctly, or credentials may be invalid, while the UI still shows success.

This project was created to deeply understand and solve this problem. The core idea was simple but strict:

> A workflow must never show success unless real backend actions actually happened.

This single principle influenced every architectural and design decision in the project.

1.2 Scope of This Documentation

This document covers:

- The complete system design
- Backend and frontend architecture
- Workflow execution logic
- Integration with Gmail OAuth and Telegram
- AI Agent integration
- Docker-based deployment and debugging
- Major errors and how they were solved
- Learnings gained during development

This is a **full-scale technical documentation**, not a short summary or abstract.

FlowZen (also referred to as in early phases) is a custom-built, AI-first workflow automation platform inspired by tools such as n8n. The system is designed to allow users to visually create workflows using nodes, connect those nodes to define execution order, and run those workflows reliably using a strong backend execution engine.

Unlike low-code tools that hide execution logic, this project focuses heavily on correctness, transparency, and backend reliability. Every workflow execution is validated strictly so that no workflow can appear successful unless real actions are executed in the backend.

The platform supports real-world integrations such as Gmail (using OAuth, not SMTP), Telegram bots via webhooks, background execution using Celery, credential management, logging, and AI Agent nodes powered by a local LLM.

This documentation provides a **complete and detailed explanation** of how the project was designed, built, debugged, stabilized, and validated, written in very simple English so that even beginners can understand it.

---

2. Problem Statement

Building a workflow automation platform is extremely complex because it combines multiple difficult systems into one product. These systems include:

- Visual user interfaces
- Backend execution engines
- Background workers
- External APIs and webhooks
- Credential storage and security
- Databases and migrations
- Deployment using containers
- AI systems with non-deterministic behavior

If any one of these systems fails, the entire platform becomes unreliable.

During the development of this project, many serious problems were encountered. Some of the most critical problems included:

- Workflows appearing connected in the UI but having no real execution graph in the backend
- Execution engine marking workflows as COMPLETED even when no real action node executed
- Gmail OAuth working in backend tests but failing when workflows were run from the UI
- Telegram webhooks returning errors or not triggering execution
- Credentials appearing valid in the UI but failing at runtime
- Database models and database schema becoming out of sync
- Docker containers failing silently or restarting repeatedly
- Celery workers failing to connect to Redis
- AI Agent nodes responding with generic or incorrect output due to bad input mapping

These problems made it clear that simply making a UI that "looks correct" is not enough. Automation platforms must be **strict, transparent, and honest** in how they execute workflows.

This project directly addresses these challenges by enforcing validation at every layer of the system.

2.1 Real-World Impact of These Problems

If such problems occur in real automation platforms:

- Businesses may lose leads because emails are not sent
- Customers may not receive important notifications
- Duplicate messages may be sent, causing spam
- Operators may not understand what went wrong

Therefore, solving these problems is not optional. It is essential.

Modern automation systems often fail silently. Many tools show success messages in the UI even when backend execution fails. This creates serious problems such as:

- Emails not being sent even though the UI shows success
- Webhooks triggering but no actions executing
- Credentials appearing valid but failing at runtime
- Duplicate executions or missing executions
- Systems working in testing but failing in real usage

During development of this project, the following major problems were faced:

- Visual workflows looked correct, but backend execution did not match
- Workflows were marked as COMPLETED even when only trigger nodes ran
- Gmail OAuth worked in isolated backend tests but failed in UI workflows
- Telegram webhooks returned errors or did not trigger execution

- Database schema and Django models went out of sync
- Docker containers crashed or restarted continuously
- Celery workers failed due to Redis or schema issues
- AI Agent nodes responded with generic or incorrect output

These issues clearly showed that automation platforms must be built with strict validation and deep observability. This project directly addresses these problems.

---

3. Project Objectives

The objectives of the FlowZen project were not limited to just building a working automation tool. Instead, the goals were defined at multiple levels: technical, architectural, learning-based, and production-oriented. This multi-layered objective setting is what made the project large, detailed, and realistic.

3.1 Technical Objectives

The technical objectives focused on building a system that behaves correctly under real-world conditions. These objectives included:

- Designing a workflow execution engine that strictly follows graph-based execution rules
- Ensuring that workflows cannot execute in an invalid state
- Preventing trigger-only executions from being marked as successful
- Making sure that every action node execution is tracked and validated
- Supporting background execution using Celery without losing execution state
- Enforcing strict validation at runtime, not just at save time

Each of these objectives was chosen because automation systems often fail at runtime rather than at design time.

3.2 Integration Objectives

Another major objective was to integrate real external systems instead of mock services. This included:

- Sending real emails using Gmail OAuth instead of SMTP
- Receiving and responding to real Telegram messages using webhooks
- Handling OAuth tokens securely and refreshing them correctly
- Ensuring that credentials are always user-scoped and never global

These objectives ensured that the platform interacts with real services exactly as a production system would.

3.3 Reliability Objectives

Reliability was a core objective throughout the project. The system was designed so that:

- No silent failures are allowed

- Errors must fail fast and be logged clearly
- Execution history must always reflect what actually happened
- Background worker crashes must not corrupt execution state

This focus on reliability separates this project from simple demo applications.

3.4 Learning and Engineering Objectives

From a learning perspective, the project aimed to:

- Understand how large automation platforms are built internally
- Learn how execution engines work beyond UI-level abstractions
- Gain hands-on experience with OAuth, webhooks, Docker, and Celery
- Practice debugging production-style errors using logs instead of guesswork

This combination of learning and production goals resulted in a much deeper project.

The primary objectives of this project are:

- To build a **node-based visual workflow automation platform**
- To ensure **UI representation always matches backend execution**
- To prevent **fake or silent success states**
- To support **real integrations** (Gmail OAuth, Telegram bots)
- To enforce **strict credential validation and scoping**
- To support **background execution with reliability**
- To integrate **AI Agents** in a controlled and explainable way
- To make the system **debuggable using logs and execution history**

Every design decision in the project prioritizes correctness over shortcuts.

---

4. System Overview

At a high level, the system works as follows:

1. A user creates a workflow using a visual builder
2. The workflow is saved as a graph containing nodes and edges
3. A trigger node starts execution (Webhook, Telegram, Manual)
4. The backend execution engine validates the workflow
5. Nodes are executed one by one in correct order
6. Background tasks are processed using Celery workers
7. Execution logs and results are stored in the database
8. The UI shows real execution status based on backend data

This flow ensures that the system behaves predictably and transparently.

---

5. Technologies Used

- **Backend Framework:** Django
- **API Layer:** Django REST Framework
- **Frontend:** Vanilla JavaScript, HTML, Tailwind CSS
- **Workflow Engine:** Custom DAG-based execution engine
- **Background Processing:** Celery
- **Message Broker:** Redis
- **Database:** PostgreSQL (production-style), SQLite (local)
- **Email Integration:** Gmail API using OAuth
- **Messaging Integration:** Telegram Bot API (webhooks)
- **AI Engine:** Local LLM via Ollama
- **Containerization:** Docker and Docker Compose

---

6. Architecture Explanation (Simple)

Nodes and Edges

- Nodes represent actions such as Trigger, Email, Telegram Send, AI Agent, Delay
- Edges represent real execution connections between nodes
- Edges are the **only source of truth** for execution order

Execution Rules

- Every workflow must have at least one trigger node
- At least one action node must execute successfully
- Trigger-only execution is rejected
- A workflow cannot be marked COMPLETED unless actions actually run

These rules ensure execution honesty.

---

7. Detailed Development Phases (Clean Unified Timeline)

The FlowZen project evolved through multiple realistic engineering phases. The development was iterative, meaning that features were built, tested, broken, debugged, and stabilized repeatedly. The source code contains evidence of this evolution through multiple model versions, migration fixes, audit scripts, and debugging utilities.

Below is the unified and cleaned development timeline.

Phase 1: Foundation and Core Architecture

The project began with a Django backend. The first major goal was to represent workflows as graphs.

Key work included:

- Workflow model storing graph JSON
- Node and edge representation
- Execution tracking models

Multiple schema iterations were created to reach a stable execution design.

Phase 2: Execution Engine Development and Validation

Once workflows could be stored, the execution engine was implemented.

Responsibilities of the engine:

- Identify trigger nodes
- Traverse graph edges
- Execute nodes in correct order
- Persist execution state

A strict rule was enforced:

> A workflow cannot succeed unless at least one action node runs successfully.

Phase 3: Credential System and Security Enforcement

A full credential subsystem was built to support Gmail, Telegram, and future integrations.

Fixes applied:

- User-scoped credential ownership
- Uniqueness enforcement per credential type
- Runtime validation
- Cleanup utilities for duplicates

Phase 4: Gmail OAuth Email Integration

Email sending was implemented using Gmail API OAuth.

Key components:

- Refresh token persistence
- Access token renewal
- Sender validation nSMTP fallbacks were removed for safety.

Phase 5: Telegram Webhook Automation

Telegram bot workflows were integrated using webhooks.

Key learnings:

- Telegram triggers cannot be executed manually
- URL routing must be exact
- Runtime credential checks are mandatory

Telegram Trigger → AI Agent → Telegram Send became a working execution flow.

Phase 6: Background Execution with Celery and Redis

Celery workers and Redis were integrated for background execution.

This enabled:

- Long-running workflows
- Delays and scheduled tasks
- Webhook-driven asynchronous execution

Schema mismatches and worker crashes were resolved using safe migrations.

Phase 7: AI Agent Node Integration

An AI Agent node was integrated using a local LLM via Ollama.

Challenges included:

- Input mapping correctness
- Expression evaluation consistency
- Preventing generic AI responses

This added significant AI-first depth to the automation engine.

Phase 8: Verification, Auditing, and QA

The project includes audit and QA scripts that simulate real user journeys and detect execution edge cases.

This ensures the system is:

- Debuggable

- Reliable
- Production-style

---

8. Detailed Module-Wise System Explanation (Expanded)

To make FlowZen a complete automation platform, the project was divided into multiple internal modules. Each module has a clear responsibility. This section explains each module in a very detailed way.

8.1 Visual Workflow Builder Module

The Workflow Builder is the frontend system where users create workflows visually.

Key responsibilities:

- Allow users to drag and drop nodes
- Allow users to connect nodes using edges
- Allow users to configure node properties
- Save the workflow graph as JSON

Important UI concepts:

- Nodes represent actions or triggers
- Edges represent execution order
- The canvas is only a visualization of backend logic

A major bug was fixed where workflows looked connected but had no real edge data. After fixing this, the UI became consistent with backend execution.

8.2 Workflow Storage and Graph Representation

Workflows are stored as structured graphs.

Each workflow contains:

- Node list
- Edge list
- Node configuration
- Metadata

This design allows workflows to be executed repeatedly without rebuilding them.

8.3 Execution Engine Module

The Execution Engine is the most important backend module.

Its job is to:

1. Load workflow graph
2. Validate correctness
3. Identify trigger node
4. Traverse edges
5. Execute nodes step-by-step
6. Store execution results

Execution is strict and deterministic.

A workflow cannot succeed unless at least one action node runs.

8.4 Node Ecosystem Module

FlowZen supports multiple node types.

Examples include:

- Trigger Nodes (Webhook, Telegram Trigger)
- Action Nodes (Send Email, Telegram Send)
- Logic Nodes (Delay, Condition planning)
- AI Nodes (AI Agent)

Each node has:

- Input schema
- Execution logic
- Output schema nThis modular design makes the system extendable.

8.5 Credential and Security Module

Credentials are stored securely and scoped per user.

Key rules:

- One credential per user per type
- Runtime validation required
- Invalid credentials fail execution

Credential handling was one of the hardest engineering parts of the project.

8.6 Gmail OAuth Email Module (Deep Explanation)

Email sending uses Gmail API OAuth.

Steps involved:

1. User connects Gmail account
2. OAuth tokens are stored
3. Refresh tokens persist
4. Access tokens are renewed
5. Email is sent through Gmail API

SMTP fallbacks were removed to ensure security.

8.7 Telegram Automation Module (Deep Explanation)

Telegram automation works using webhooks.

Lifecycle:

- User sends message to bot
- Telegram sends webhook POST
- Trigger node receives input
- Execution engine starts workflow
- AI Agent generates response
- Telegram Send node replies

Manual testing is not possible for webhook triggers.

8.8 Background Processing Module (Celery + Redis)

FlowZen supports background execution.

Celery handles:

- Long-running tasks
- Scheduled tasks
- Webhook-driven async execution

Redis is used as the broker.

Worker crashes and schema mismatches were debugged and fixed.

8.9 Logging and Observability Module

Every workflow execution generates logs.

Logs store:

- Node execution details
- Errors and traces
- Execution timestamps

This makes the system highly debuggable.

8.10 Verification and QA Module

The project includes verification scripts.

These scripts:

- Simulate user workflows
- Detect schema mismatches
- Ensure end-to-end correctness

This level of QA is uncommon in basic projects.

---

9. Expanded Final Outcome

FlowZen is now a stable automation platform foundation with:

- A working visual workflow builder
- A strict backend execution engine
- Gmail OAuth email delivery
- Telegram webhook automation
- Background task execution with Celery
- AI Agent integration
- Reliable logs and execution history

The platform behaves honestly: it never shows success unless real actions execute.

FlowZen is now a stable automation platform foundation with:

- A working visual workflow builder
- A strict backend execution engine
- Gmail OAuth email delivery
- Telegram webhook automation

- Background task execution with Celery
- AI Agent integration
- Reliable logs and execution history

---

10. Step-by-Step Workflow Execution Example (Very Detailed)

To understand FlowZen clearly, it is important to see how a real workflow executes from start to finish.

This section explains a complete example workflow in very simple and detailed steps.

Example Workflow: Telegram User Message → AI Reply → Telegram Response

Workflow Structure:

1. Telegram Trigger Node
2. AI Agent Node
3. Telegram Send Node

Step 1: User Sends a Message

A user opens Telegram and sends a message such as:

"Hello, what is automation?"

This message is sent to the FlowZen Telegram Bot.

Step 2: Telegram Webhook Fires

Telegram does not wait for FlowZen to ask for messages. Instead, Telegram pushes the message automatically using a webhook.

Telegram sends an HTTP POST request to:

`/api/webhooks/telegram/`

This request contains:

- User ID
- Chat ID
- Message text
- Timestamp

Step 3: Trigger Node Seeds Workflow Input

The Telegram Trigger node receives this webhook payload.

It extracts the clean user message and converts it into structured workflow input.

Example input data:

- clean_text = "Hello, what is automation?"
- sender_id = Telegram user

Step 4: Execution Engine Starts Workflow Run

Once trigger input is available, the execution engine:

- Creates a WorkflowExecution record
- Marks status as RUNNING
- Identifies next node using edges

Step 5: AI Agent Node Executes

The AI Agent node receives the user message.

It sends the input to the local LLM through Ollama.

The AI model generates a response such as:

"Automation means making tasks happen automatically without manual effort."

The AI output is stored as node output.

Step 6: Telegram Send Node Executes

The Telegram Send node takes the AI response text.

It uses the stored Telegram Bot credential and calls Telegram API:

- chat_id
- message_text

Telegram delivers the response back to the user.

Step 7: Execution Completes Successfully

After all nodes execute:

- WorkflowExecution status becomes COMPLETED
- Logs are stored
- UI displays real success

FlowZen only shows success because real Telegram delivery occurred.

---

11. Expanded Error Case Studies (Detailed)

This section explains major errors as real debugging stories.

Case Study 1: Workflow Marked Completed Without Actions

Problem: Workflows were showing COMPLETED even when only trigger nodes ran.

Root Cause: Execution engine exited early without validating action execution.

Fix: Strict rule enforced: at least one action node must run.

Learning: Automation platforms must never allow fake success.

Case Study 2: Gmail OAuth Appeared Successful But Emails Never Arrived

Problem: Deploy and test showed success, but inbox received nothing.

Root Cause: Refresh tokens were not stored properly.

Fix: Refresh token persistence and strict runtime validation.

Learning: OAuth is only correct if real delivery happens.

Case Study 3: Telegram Webhook Returned 404

Problem: Telegram could not reach the webhook endpoint.

Root Cause: URL routing was missing in Django urls.py.

Fix: Webhook path registered correctly.

Learning: Webhook systems require exact routing.

Case Study 4: Docker Containers Restarting Continuously

Problem: Backend crashed repeatedly inside Docker.

Root Cause: Multiple conflicting Docker configs and missing imports.

Fix: Single Dockerfile locked, full cleanup performed.

Learning: Infrastructure problems can hide real code bugs.

---

12. Security Considerations (Expanded)

Automation platforms deal with sensitive data.

FlowZen applies important security principles:

- OAuth tokens are stored securely
- Credentials are scoped per user
- No global credential leakage
- Execution fails if credentials are invalid
- Webhooks are treated as trusted entry points

Security is essential because automation systems can access emails, chats, and APIs.

---

13. Node Catalog (Expanded)

FlowZen supports a growing ecosystem of nodes.

Current implemented nodes include:

- Telegram Trigger
- Telegram Send
- Gmail Send Email
- AI Agent Node
- Manual Trigger

Planned logic nodes include:

- Delay
- Condition
- Scheduler

- Loop

This modular node design allows unlimited expansion.

---

14. Deployment and Local Development Setup (Detailed)

FlowZen was designed to run like a production system locally.

Components:

- Django backend server
- PostgreSQL database
- Redis broker
- Celery worker
- Celery beat scheduler
- Docker Compose orchestration

Development required careful debugging of:

- Port conflicts
- Worker startup order
- Migration mismatches

This created a realistic engineering environment.

---

15. Final Conclusion (Expanded)

FlowZen is a large-scale AI-first workflow automation platform built with strong backend engineering principles.

The project demonstrates:

- Real workflow execution logic
- Honest success/failure reporting
- OAuth-secure Gmail email delivery
- Webhook-based Telegram automation
- Background execution with Celery
- AI Agent integration using local LLMs
- Strong debugging through logs and QA scripts

FlowZen is not just a UI project. It is a true automation engine foundation that reflects production-level complexity.

This documentation represents a complete, detailed, and submission-ready explanation of the system.

End of Final Project Documentation

FlowZen is a production-style AI-first workflow automation platform built with strong engineering principles.

The project demonstrates deep understanding of:

- Workflow execution engines
- OAuth and secure credential handling
- Webhook-based triggers
- Docker and background workers
- AI-assisted automation
- Debugging through logs and verification

This document represents a clean, professional, and submission-ready final documentation of the project.

End of Final Project Documentation